

手描きスケッチからロボット形態のタスク適性を判定し修正指針を返すシステム

6 枚版・中間発表配布資料

1 要旨

ロボットアームの設計では形態と制御が相互に依存するため、両者を切り離れた「作ってから直す」開発サイクルは非効率である。本研究は、手描きスケッチから (i) シミュレーションモデルの生成、(ii) 形態のタスク適性の判定、(iii) 不適合時の修正指針の提示までを人手を介さず接続するシステムを構築する。中核には形態-制御 co-design (Stackelberg PPO) を判定エンジンとして転用する。3 タスク × 2 乱数シードの統制実験により、判定エンジンがタスクの要求に応じて異なる基準で形態を評価すること、幾何だけでは区別できない形態対を判別できること、返す修正指針が実際にタスク達成へつながることを示す。あわせて、指針の実行可能性は判定の正しさとは独立に壊れうること、重み転用が判定に要する学習量を削減すること、および固定根本アームという新しい実験系へ co-design を移植する過程で顕在化した設計上の落とし穴（報酬の複雑化・報酬シェーピングの初期値依存・探索範囲の境界と比較可能性）を報告する。

2 はじめに

ロボットアームの設計では形（形態）と動かし方（制御）が相互に依存するため、「この形が良いか」は制御を仕上げてみるまで分からない。試作と検証のコストが高く、既存技術には 2 つの穴が残る。第一に、スケッチや 3D 生成メッシュには可動構造が入っていない。どこが関節でどこがリンクかは形状生成技術の射程外である。第二に、形が作れてもタスクに向くかは分からない。形状生成の評価は「作る」までで、動くロボットとして妥当かの評価は扱われない。結果として現状は「作ってから直す」しかなく、本研究はこの手順を反転させる。

本研究の主軸はシステムであり、co-design はその中核部品にすぎない。co-design を中核に据える理由は分離不能性にある。形態の補正はタスク性能を基準にしてのみ方向づけられ、タスク性能は形態が定まって初めて測れるため、両者を独立には解けない。本稿では、このシステムの中核である判定エンジンの妥当性と修正指針の実行可能性を統制実験によって示し、あわせて重み転用による判定の高速化、システム全体の費用構造、および新しい実験系への移植で得た設計知見を報告する。

3 関連研究

本研究の関連領域は 4 系統に大別される。スケッチによる三次元モデリング (Teddy, RobotSketch 等) は形状生成が主眼で、物理的整合性の判断や、生成形態がタスクに向くかの判定はユーザに委ねられている。三次元形状からの可動構造推定 (URDF-Anything 等) は点群からの学習ベース推定であり、密な関節構造では推定誤差が有限確率で残存し、誤りが正常な出力と区別できない。形態-制御 co-design (Sims 1994 以来の系譜、Transform2Act、本研究が基盤とする BodyGen・Stackelberg PPO) は、より良い形態を自動設計する手法として発展しており、タスク間の形態比較や、判定に必要な学習量の較正は評価の対象になってこなかった。タスク適合性の判定と設計へのフィードバックについては、到達可能性解析や可操作性 (Yoshikawa, 1985) が形態の原理的能力を測るが、制御がその能力を実際に引き出せるかは測らない。反実仮想説明・実行可能な救済 (Wachter et al., 2018; Ustun et al., 2019) は学習済み分類器を対象に判定を覆す最小変更を最適化的に探索するが、本研究の第 1 層は幾何が支配する範囲に限り運動学から解析的に必要倍率を導く点で異なる。

本研究の新規性は個々の観点の改善ではなく、独立に発展してきた 3 領域（スケッチからの形状生成・強化学習による co-design・反実仮想説明に基づく設計フィードバック）を、専門知識を持たない設計者が使える単一パイプラインとして一貫通貫に結合したことにある。結合には 2 つの断絶がある。形状生成の出力は可動構造を持たない単一メッシュであり co-design の入力形式と隔たりがあること、co-design の出力はスカラーのスコアと収束パラメータであってそのままでは行動可能な情報ではないことである。前者は関節マーカー規約で、後者は三層診断で埋める。

4 提案システム

システムは7つのモジュールからなる。M1: 画像整形（関節位置にマゼンタ球を描かせる規約を課す）、M2: 3D 形状生成、M3: 色検出によるリンク分割と関節特定、M4: 形態パラメータ抽出、M5: シミュレーションモデル生成、M6: co-design による判定、M7: 三層形態診断。可動構造の欠落に対しては、点群からの学習ベース推定に頼らず**入力側に規約を課す**。構造抽出を確率的推定から閾値つき色検出という決定的処理に置き換えることで、検出失敗が明示的に検出可能になる。

ただし決定的処理が頑健性を自動で保証するわけではない。M1・M2 は生成 AI に依拠し出力は確率的である。生成 AI が出力するマゼンタは指定値から最大 86/チャンネルずれ、公称色に固定しきい値をかけると 2 メッシュとも検出に失敗する。しきい値を手で広げれば検出はできるが必要な値がメッシュごとに異なり、両立する固定値のもとでは関節位置が最大 12.8 mm ずれる。基準色をメッシュごとに推定する処理を前段に置いて初めて誤差 1 mm 以内に収まった。**システムの頑健性は個々のアルゴリズムの決定性ではなく、外部サービスの出力ブレをどこで吸収するかという設計で決まる。**

システムは4つの軸で評価する。入力からモデルが確実に作れるか、中核の判定エンジンは妥当か、返した助言は使えるか、システムとして何秒/何時間で回るか。が本稿の主題であり、山場は「タスクごとに判定基準が変わるか」 (§4) と「返した助言は本当に使えるのか」 (§6) の2点である。

5 判定エンジンの妥当性: 前提の確認

タスク間の比較に入る前に、そもそも形態最適化が学習に寄与しているかを確認しておく必要がある。co-design が形態を賢く選んでいるように見えても、実は制御学習だけで同じ性能に達するなら、形態最適化を評価する意味がない。Pusher タスクで、形態最適化を有効にした条件 (B) と、形態更新の歩幅を実質ゼロにして形態を凍結した条件 (C) を比較したところ、キューブ前進速度は B が 0.10~0.12 に到達したのに対し C は 0.01~0.02 に留まった (約 8~10 倍の差)。**形態最適化なしでは本実験系の制御学習は実用的な水準に到達しないことを確認したうえで、以降の判定エンジンの検証に進む。**

6 判定エンジンの妥当性: タスク間の識別

6.1 タスク識別

タスク以外の条件を揃え、3 タスク (Pusher: 押す / Reach: 届く / Target-Pusher: 狙って止める) × 各 2 シードで比較した。Pusher は肘ギア比が両シードとも探索上限 400、Reach は肩ギア比が両シードとも探索下限 20 に収束し、Target-Pusher は Pusher 寄りの高ギアに収束する。行動もタスクごとに分離し (Pusher は一撃で吹き飛ばす、Reach はサブミリ精度で静止、Target-Pusher は押して止める、誤差 0.5 cm)、分離は個別パラメータではなく機能量 (ギア比プロファイル・リーチ帯・行動類型) の水準で現れ、2 シードを跨いで一致する。

なぜこの向きに分かれるか。 Pusher が肘ギア上限を選ぶのは、速度報酬の総和が最終移動距離に比例するため、最適戦略が一撃で最大の運動量を与えること (打撃的方策) に帰着するからである。Reach が肩ギア下限を選ぶ理由は直感に反する。決定論的な到達精度自体はギア比に依存しない (肘ギア 207 でもサブミリ着地は可能)。効いているのは**確率的方策のもとでのノイズ**で、高ギアは行動ノイズを増幅し誤った位置への整定を引き起こす (n=10 で 5/10 が 23~38 mm の誤着地)。低ギアの利得は精度ではなく**ノイズ下の頑健性**であり、学習が確率的ロールアウトで進む以上、Leader は頑健な低ギア形態を選好する。この解釈は評価方式にも波及する。Reach の 2 シードは決定論評価では高ギア側が優位だが、確率的評価では低ギア側が 3.6 倍優位になり**評価方式によって優劣が逆転する**。本システムの判定が確率的ロールアウトの学習曲線に基づくのは、この逆転を踏まえた設計である。

6.2 外部形態の順位付け (前向き判定)

システムが実際に行うのは「形を与えて順位を付ける」ことなので、これも直接試した。rrbot (手作りモデル) で目標 0.8 m に対し届かない短腕 (0.55 m) と届く長腕 (約 1.0 m) を Reach で比較すると、長腕 (best -4.85) が短腕 (best -250.02) を明確に上回った。同じ判定をパイプライン生成形態 (0.5 倍・1.8 倍にスケール) でも行い、届く長腕 (best -12.48) が届かない短腕 (best -396.92) を上回ることを確認した。ただしこちらは順位が確定するまで **ep27** を要しており、rrbot の「ep0 で確定」は一般化しない。判定を早期に打ち切ると良い形態と悪い形態を取り違える。

6.3 タスク間の順位反転（山場 1）

「結局『長い腕ほど良い』と言っているだけでは」という批判に答えるため、同じ 3 形態（総リーチ 0.403 / 0.887 / 1.451 m）を 2 タスクにかけた。

形態（総リーチ）	Reach	Pusher
短腕 0.403 m	-396.92（3 位）	-0.00（3 位）
中間 0.887 m	-3.42 ~ -1.66（1 位）	34.32 ~ 44.94（2 位）
長腕 1.451 m	-12.48 ~ -10.27（2 位）	96.18 ~ 99.37（1 位）

中間と長腕の順位が入れ替わり、**両タスクとも 2 シードの範囲が重ならない**。Reach は目標距離への適合を要求するため余った長さは慣性と姿勢の自由度になり不利、Pusher は先端速度（角速度 × リーチ）を要求するため長さがそのまま有利になる、という読みと整合する。判定器は形の絶対的な大きさではなく、**タスクの要求に照らして形を評価している**。

6.4 判別解像度の拡張

総リーチを 0.887 m に固定しリンク長の配分だけを反転させた 2 形態（根元重／先端重）は、最大リーチが同一のため幾何診断では原理的に区別できない。co-design は Reach で根元重を先端重より 2.4 ~ 3.5 倍高く評価し（-1.65 ~ -1.81 対 -4.39 ~ -5.75、範囲非重複）、**幾何の外側を co-design が埋めている**具体例となった。ただし Pusher で同じ比較をすると差は 1.03 ~ 1.49 倍に縮み、均等配分（34.32 ~ 44.94）と区別できなかった。**配分の効き方はタスクに依存する**。

加えて、可動平面自体を向け直せる非平面 4 関節アームでも Reach/Pusher の識別は独立に再現し、先端リンクの太さが Reach で下限 0.030 m、Pusher で上限 0.100 m に分岐した（2 シードとも 4 本全て探索範囲の端）。ただし Pusher の到達スコアはシード間で 2.1 倍開く（124.36 対 263.14）ため、**識別の成立は主張するが到達性能の水準は主張しない**。また非平面モデルはパイプライン出力ではなく手作業で構成したものであり、これが示すのは**判定エンジンの一般性であってパイプラインの一般性ではない**。

7 修正指針とその実行可能性（山場 2）

判定が返すスカラーだけでは設計者は次の一手を決められない。そこで幾何・収束設計・行動の三層診断を実装した。第 1 層は学習を実行せず約 1 秒で不適合を検出し、判定を覆すのに必要な拡大倍率を数値で返す。マトリクス 6 条件に対し総合判定は実測順位と 6/6 一致した。この第 1 層は到達可能性解析（Zacharias et al., 2007）の最小形であり手法として新規性はなく、新規なのはこれを co-design の**前段スクリーニング**として置き、学習後の情報（収束設計・実際の行動）と階層的に組み合わせる点にある。

三層に分ける根拠は事後検証で裏付けられた。第 1 層が区別できない形態対（Reach・Pusher いずれも中間スケール対長腕）に対し、第 3 層（行動の実測量）は実測順位と同じ向きの結果を返した（Reach: 最終誤差 0.9 mm 対 7 mm、Pusher: 対象の移動距離 1.48 m 対 4.05 m）。一方、**第 2 層（収束した設計パラメータの境界張り付き）は単独では両方向に誤ることが確認された**。「ギア比も太さも探索下限に張り付く」という署名は Reach の最短形態（失敗）と中間スケール（1 位）の双方に現れ、逆に Pusher の最短形態では設計パラメータが範囲の内側で収束していながら対象に接触すらできていない。**第 2 層は単独の判定根拠としては用いない**というのが本研究の結論である。

7.1 閉ループで初めて見つかった 2 つの欠陥

不適合形態（総リーチ 0.403 m、目標 0.800 m）に第 1 層が返した倍率をそのまま適用し、生成 → 再判定まで一度通してみたところ、2 つの欠陥が露呈した。第一に、必要倍率 1.9846 が小数第 2 位への丸めで 1.98 に切り捨てられ、助言どおりに直しても 2 mm 足りずに再度却下された（切り上げに修正）。第二に、修正版の助言を検証する過程で、形態生成器がリンクの描画長のみを倍率倍し関節の取り付け位置を据え置いていたため、公称総リーチ 0.802 m に対し実効リーチが 0.5038 m（63%）しかないことが判明した（生成器を修正）。いずれも既存形態に診断をかける検証では原理的に検出できず、助言を適用した形を実際に作って再入力するまで見えなかった。判定が正しいことと、助言が使えることは別物である。

7.2 閉ループの実効性

是正後、不適合形態（平均残存距離 397 mm）に助言の下限倍率を適用しただけで残存距離は 0.33 mm/0.27 mm（2 シード）まで改善した。助言に従えば、達成できなかった形が達成できる形になることを実証した。

7.3 余裕を持たせる設計判断は実験に否定された

下限ちょうどでは腕を伸ばしきった特異姿勢でしか目標に触れられないため、当初は余裕 10% を推奨値とする設計にした。しかし余裕 0%・5.3%・10% の三点比較（各 2 シード、隣接条件も範囲非重複）で残存距離は 0.27～0.33 mm、1.19～1.41 mm、1.66～3.42 mm と単調に悪化する。制御コスト係数を 0 にしても差は残るため、この悪化は制御コストでは説明できない。一方 Pusher では下限（29.25）より余裕 10%（34.32）の方が良く、必要な余裕はタスクに依存し、一律の推奨値には根拠がない。反実仮想的な助言を返せるのは幾何の層のみであり、「届くが遅い」には原因を報告できても直し方は返せない、という限定も付す。

8 重み転用による判定の高速化

判定を実用的な時間で返すには、学習量の削減が要る。学習済み重みの転用のうちのどの成分が判定を速めるかを、制御方策ネットワークのみ・形態ネットワークのみ・全成分の 3 条件（Pusher タスク、転用元 best 140.47）で分解した。

転用条件	best	転用元との比較
制御方策のみ（K1）	93.82	転用元を下回る
形態のみ（K2）	193.69	転用元を上回る
全成分（I1）	210.58	転用元を最も上回る

形態ネットワークを含む転用（K2・I1）が転用元自身の到達点を上回った一方、制御方策のみの転用（K1）は下回った。制御方策の重みは既に収束済みで可塑性が低く、動き続ける形態に追従できない。形態ネットワークを含む転用では Follower がゼロから学習するため可塑性が高く、早期に安定した形態へ同期できる、と解釈できる。

判定に必要な最小学習量も 11 コホートで較正した。所要エポック数は候補間の隔たりに支配され、設計が明確に異なる候補どうしは 30 エポック以内、乱数しか変わらない実質同一の候補では 120 エポックを要する。§5.2 の前向き判定（ep27 で確定）はこの較正值と整合する一方、rrbot の「ep0 で確定」は一般化しない実測例として位置づけられる。

9 システムとしての到達点（費用構造）

1 周あたりの費用は二極化している。

工程	所要
モデル生成・幾何診断・三層診断	合計数秒（学習不要）
判定（設計が明確に異なる候補）	約 4 時間（30 エポック）
判定（候補が拮抗する場合）	約 16 時間（120 エポック）

幾何的に破綻した形は学習を待たずに数秒で棄却して直し方を返せる。実際にマトリクスの 6 条件中 2 条件は学習なしで処理された。明らかに届かない案を何度描き直しても費用がほとんど増えない構造は、概念設計段階の支援として機能する。

10 環境移植で得られた設計知見

Stackelberg PPO の原論文が対象とするのは自由関節を持つ移動可能なロボットであり、本研究が用いる据え置き型の固定根本アームへ移植する過程で、co-design を新しい実験系へ適用する際に踏みやすい 3 つの落とし穴が顕在化した。いずれも修正には数値ではなく設計方針の転換を要した点で、単なるバグとは性質が異なる。

報酬の複雑化は探索信号そのものを消しうる。打撃的方策を望ましくない挙動とみなして排除しようとする報酬改造（目標座標化・シェーピング・到達ボーナスの追加）を試みたが、いずれも別の意図せぬ局所解を誘発するか、探索信号そのものを消失させた。打撃的方策は速度報酬に対する**正当な最適解**であり、排除すべき対象ではなかった。この現象が co-design 特有である理由は、Leader の勾配が Follower 側の損失の Hessian の逆作用を含む構造にある。報酬の複雑化は形態の評価関数を歪めると同時に、Follower の最適応答を不安定にして Hessian の条件数を悪化させ、Leader は歪んだ評価関数を劣化した勾配で最適化することになる。単に方策の学習が難化するだけの通常の強化学習とは異なり、**形態探索の駆動力そのものが失われうる**。結論として、単一項の単純な報酬を保ち、タスクの定義自体を変えることが正しい介入である。

報酬シェーピングの初期値は形態に依存させてはならない。ポテンシャルベース報酬シェーピング (PBRs) の方策不変性は形態が固定された制御方策に対して証明されたものであり、co-design ではエピソード合計が「最終ポテンシャル - 初期ポテンシャル」に潰れるため、初期ポテンシャルが Leader の決める形態に依存すると「対象から遠くに生まれる形態ほど報酬の伸び代が大きい」という歪みが構造的に生じる。実際に形態が対象から遠ざかる方向へ成長する現象が繰り返し観測された。対策は、シェーピングに用いるポテンシャルを形態に依存しない量に限ることである。

探索範囲の境界への収束は、タスクの選好と探索予算の産物を区別しないと誤読する。最適形態が設計空間の境界で決まってしまう設定では、その形態はタスクの選好ではなく探索範囲の大きさを測っているに過ぎない。実際に初期世代の Reach 実験は目標 (1.5 m) が到達可能範囲 (最大 1.44 m) の外にあり、形態が探索範囲の上限に張り付く「幾何学的床」が生じてタスク比較として解釈不能になった (決定論方策の残距離 6.02 cm が 1.5-1.44 と一致することで確認)。是正は目標を到達可能範囲の内側 (0.8 m) へ移すことであり、是正後の形態は境界に接しない内部解へ収束した。**形態比較の実験は、各タスクの理論最適が設計空間の内部で分かれることを事前に確認して設計しなければならない**。

11 考察と限界

本研究が扱うのは、量産を見据えた詳細設計に先行する概念設計段階、すなわち設計案がそもそもタスクを達成しうるかを見極める最上流の工程である。この工程で得られた知見は次の 3 点に集約される。第一に、co-design を判定器として用いる際は、収束した設計パラメータの境界値への収束という一見有力な不適合の指標が単独では成否を判別できない (§5.4 の Reach 署名は成功・不成功の両方に現れ、§6 の第 2 層検証も同じ結論を独立に支持する)。幾何・設計・行動を分けて読む必要がある。第二に、反実仮想的な助言を返すシステムは、出力の正しさとは別に、助言の値・助言に従った形態の生成・再入力のをそれぞれを独立に検算する手順を持たなければならない (§6.1)。この欠陥は判定の正しさを検証する枠組みからは原理的に見えない。第三に、判定器の設計判断 (余裕を持たせる等の安全側の直感) はタスクごとに検証を要し、一律の推奨には根拠がない (§6.3)。

限界: 各条件 1~2 シードであり統計的検定は行っていない。非平面形態の検証に用いたモデルはパイプラインの出力ではなく手作業で書き起こしたもの (2026-09-02 にパイプラインへ縦型モードを実装し同型の構造を生成できるようにしたが、その出力での学習は未完)。シミュレーションのみで実機検証・sim-to-real はスコープ外。人手ゼロと言えるのは関節検出以降で、画像整形と 3D 生成は Web インタフェース経由の人手が残る。利用者実験は行っておらず、「専門知識がなくても使える」という主張はしていない。手描き線画を入力とする通し実行 (E2E) は 1 例のみ実施した (2026-09-01。関節検出 3/3・公称 = 実効一致・幾何診断まで通り、描いた比率が XML まで保存されることも確認)。ただし前段の頑健性を成功率で報告できる件数ではなく、co-design 判定まで含めた通し実行も未完である。加えて 2 つの限界を追記する。**入力スケールの不整合:** 判定対象の形態の大きさは生成モデルの出力に依存して制御できない一方、タスク側の目標位置は絶対座標で与えられる。このスケール不整合は形態最適化では吸収できず、本研究は不整合を検出して必要な拡大倍率を返す立場を取った。**判定コストと物理設定への依存:** 打撃的方策が示す高い先端速度は現在の関節ダンピング係数・摩擦係数の設定に依存しており、実機相当の物理設定で同じ方策が現れるかは確認していない。ただし本研究の中心的主張はタスク間の相対的な分化であり、物理設定が全タスクへ対称に作用する限りこの依存は主張を損なわない。

12 おわりに

手描き形態からタスク適性の判定と修正指針の提示までを接続するシステムを構築し、中核の判定エンジンが要求仕様 (タスク条件で基準を変える判定器) を満たすこと、返す指針が実際に機能すること、重み転用が判定を実用的な時間へ縮めることを示した。システムの新規性は個々の構成要素の改

良ではなく、独立に発展してきた3領域を専門知識を持たない設計者が使える単一パイプラインとして一気通貫に結合した点にある。あわせて、co-design を新しい実験系へ移植する際に踏みやすい3つの落とし穴（報酬の複雑化・シェーピングの初期値依存・探索範囲の境界と比較可能性の混同）を記録した。これはアルゴリズムの欠陥ではなく、移植先の環境に対して報酬設計をどう合わせるかという実践知であり、co-design を別のタスクへ適用する後続の作業に再利用できる。残る最大のギャップは手描き線画からの通し実行であり、これは実験ではなく未実装機能として次の課題に位置づける。